

Niveau : III Date : 16/05/2025 Durée : 2h Nombre de pages : 4	Documents : Non autorisés Calculatrice : Non autorisée Annexe : Oui Barème à titre indicatif : 12.5+ 7.5	 ENSI ÉCOLE NATIONALE DES SCIENCES DE L'INFORMATIQUE <i>Ecole Nationale des Sciences de l'Informatique</i> <i>Université de la Manouba</i> <i>A.U. 2024/2025</i>
Enseignantes : A. HEDHILI, I. FLISS, I. AMDOUNI, I. BOUKHRIS, R. CHEBIL		Réf : ID.104 Version :01
EXAMEN Programmation Orientée Objet		

NB :
1. Il est à noter que tout au long de cet exercice les attributs des classes à définir doivent être non publics. 2. Pour le code C++, pour gagner de l'espace et du temps, les implémentations des méthodes pourront être données en même temps que leurs déclarations dans les classes.

Exercice 1 : Gestion de notifications multicanal en C++ (12.5 points)

Vous êtes en charge de développer un **module de notifications** pour une application métier qui doit pouvoir alerter ses utilisateurs via différents canaux (Email, SMS, Push mobile). L'objectif est de mettre en œuvre le **polymorphisme** en C++ et d'exploiter la classe générique `std::vector` (voir annexe) pour stocker et traiter de manière unifiée ces différents types de notifications.

La plateforme doit envoyer des alertes critiques (par exemple, rappels de rendez-vous, alertes de sécurité, promotions ciblées) selon trois canaux :

1. Email modélisé par la classe 'EmailNotification' : Ce canal permet d'envoyer des messages électroniques riches en contenu à des destinataires. Une notification envoyée par email comprend généralement deux parties :
 - Un sujet : résumé court du message.
 - Un corps de message : texte détaillé destiné à l'utilisateur.
Il est particulièrement adapté aux communications formelles ou nécessitant des explications précises.
2. SMS modélisé par la classe 'SMSNotification' : Ce canal est conçu pour transmettre des messages courts à un numéro de téléphone mobile. Chaque SMS est limité à 160 caractères ; au-delà, le message est découpé en plusieurs SMS facturés séparément. Ce canal est utile pour les alertes urgentes ou les rappels succincts.
3. Push mobile modélisé par la classe 'PushNotification' : La notification Push permet d'envoyer un message instantané à une application mobile installée sur un appareil spécifique, identifié par un Device ID. Le contenu est court, souvent limité à une ligne de texte, et s'affiche directement à l'écran de l'utilisateur. Ce canal est idéal pour les notifications rapides, silencieuses et non intrusives.

Chaque canal a son propre coût unitaire d'envoi et son propre mode d'envoi. Pour l'email plus le message est long, plus le coût augmente légèrement et le calcul se fait selon la formule suivante : $\text{Coût} = 0,015 + 0,001 \times (\text{longueur du sujet} + \text{du corps})$. L'envoi est simulé par l'affichage d'un message du type : "Envoi d'un Email à [destinataire] avec sujet '[sujet]' et contenu : [message]"

Pour le canal SMS, chaque tranche de 160 caractères équivaut à un SMS facturé et la formule de calcul du coût unitaire est : $\text{Coût} = 0,05 \times \text{partie entière}(\text{longueur du message} / 160)$. L'envoi est simulé par l'affichage d'un message du type : "Envoi d'un SMS à [numéro] : [message]"

Par ailleurs, pour la notification Push mobile le coût unitaire est fixe (soit 0,01), quelle que soit la taille de la notification. L'envoi est simulé par l'affichage d'un message du type : "Envoi d'une notification push vers le [device ID] : [message]"

Travail à faire :

1. On essaye de concevoir une hiérarchie de classes C++ fondée sur une **classe abstraite Notification**. Donnez le code de cette classe qui contient en plus du constructeur par défaut et du destructeur, les méthodes suivantes : la méthode 'send()' qui simule l'envoi d'une notification et la méthode 'cost()' qui calcule et retourne le coût unitaire d'envoi. (0.75 pt)

```
class Notification {  
public:  
    Notification() {}  
    virtual ~Notification() ;  
    virtual void send() const = 0;  
    virtual double cost() const = 0;  
};
```

2. Donnez la déclaration et l'implémentation des classes **EmailNotification**, **SMSNotification** et **PushNotification**. (3pts).

```
class EmailNotification : public Notification {  
private:  
    String recipient, subject, body;  
public:  
    EmailNotification(string& r, string& s, string& b)  
        : recipient(r), subject(s), body(b) {}  
  
    void send() const override {  
        std::cout << "Envoi d'un Email à " << recipient  
            << " avec sujet '" << subject << "' et contenu : " << body << std::endl;
```

```

    }

    double cost() const override {
        return 0.015 + 0.001 * (subject.length() + body.length());
    }
};

class SMSNotification : public Notification {
private:
    string number, message;
public:
    SMSNotification(string& n, string& m)
        : number(n), message(m) {}
    void send() const override {
        std::cout << "Envoi d'un SMS à " << number << " : " << message << std::endl;
    }
    double cost() const override {
        return 0.05 * (message.length() + 159) / 160;
    }
};

class PushNotification : public Notification {
private:
    string deviceID, message;
public:
    PushNotification(const std::string& d, const std::string& m)
        : deviceID(d), message(m) {}
    void send() const override {
        std::cout << "Envoi d'une notification push vers le " << deviceID << " : " << message <<
std::endl;
    }
    double cost() const override {
        return 0.01;
    }
};

```

3. On donne la **classe NotificationManager** dont le rôle est de centraliser la gestion de toutes les notifications. Implémentez les différentes méthodes de cette classe. (4,25 pts)

```

class NotificationManager {
public:

```

```

// une méthode qui permet d'ajouter une notification au gestionnaire
.....
// une méthode qui permet de supprimer la notification à l'indice donné
.....
// Envoie toutes les notifications
void sendAll() const;
// Calcule et renvoie le coût total de toutes les notifications
double totalCost() const;
// Destructeur : libère toutes les notifications restantes
~NotificationManager();
private:
std::vector<Notification*> notifications;
};

```

```

class NotificationManager {
public:
    void addNotification(Notification* notif) {
        notifications.push_back(notif);
    }

    void removeNotification(size_t index) {
        if (index < notifications.size()) {
            delete notifications[index];
            notifications.erase(notifications.begin() + index);
        }
    }

    void sendAll() const {
        for (auto notif : notifications) {
            notif->send();
        }
    }

    double totalCost() const {
        double total = 0;
        for (auto notif : notifications) {
            total += notif->cost();
        }
        return total;
    }

    ~NotificationManager() {
        for (auto notif : notifications) {
            delete notif;
        }
    }
}

```

```
private:
    std::vector<Notification*> notifications;
};
```

4. Ecrivez un programme pour tester vos classes : (1.5 pt)

- Créer dynamiquement au moins 2 emails, 2 SMS et 1 push.
- Instancier un objet NotificationManager.
- Ajouter chaque notification à ce gestionnaire.
- Déterminer et afficher le coût total de toutes les notifications gérées.

```
int main() {
    NotificationManager manager;
    manager.addNotification(new EmailNotification("user1@example.com", "Rappel", "Votre rendez-vous
est prévu à 10h.));
    manager.addNotification(new EmailNotification("user2@example.com", "Alerte", "Sécurité renforcée
dans votre zone.));
    manager.addNotification(new SMSNotification("123456789", "Code de vérification : 1234.));
    manager.addNotification(new SMSNotification("987654321", "Votre solde est insuffisant.));
    manager.addNotification(new PushNotification("device_001", "Nouvelle mise à jour disponible !));
    manager.sendAll();
    std::cout << "Coût total des notifications : " << manager.totalCost() << " €" << std::endl;
    return 0;
}
```

5. Quelle est la différence entre une méthode virtuelle et une méthode virtuelle pure ? Donnez un exemple pour chaque cas à partir de cet exercice. (0.75 pts)

- Une **méthode virtuelle** (`virtual`) est une méthode qui peut être redéfinie dans une classe dérivée, mais qui **a une implémentation par défaut** dans la classe de base. (0.25)
- Une **méthode virtuelle pure** (`virtual ... = 0`) **n'a pas d'implémentation dans la classe de base**, et **force** les classes dérivées à la redéfinir. Cela rend la classe **abstraite**. (0.25)

Exemple : 0.25

6. Est-ce que vous utilisez la liaison dynamique dans cet exercice ? Expliquez (0.5 pt)

Oui, Lorsqu'on appelle `send()` ou `cost()` sur un pointeur `Notification*`, la version de la méthode dépend du **type réel** de l'objet (Email, SMS, Push) à l'exécution.

7. Proposez (sans code) une classe qu'on peut ajouter pour faire de l'héritage multiple ? Quelles sont les modifications à faire aux codes précédents (Questions 1, 2, 3) pour intégrer cet héritage multiple ? (1.75pts)

On peut ajouter une classe `Trackable` pour permettre de **suivre l'état** ou le **journal d'envoi** d'une notification. Toute autre proposition que vous jugez correcte est possible

Exercice 2 : Gestion de Moutons dans une Ferme en Java (7.5 points)

À l'approche de l'Aïd, une ferme se prépare à commercialiser différents types de moutons. Tout mouton est caractérisé par :

- nom (*chaîne de caractères*) : identifiant du mouton.
- race (*chaîne de caractères*) : par exemple, "Barbarine", "Noire de Thibar".
- poids (*nombre réel*) : poids du mouton en kilogrammes.

La classe `Mouton` contient, **en plus de son constructeur par défaut**, les méthodes suivantes :

- `toString()` : retourne une description simple du mouton (ex. : "Nom_Mouton_ : Barbarine de 60 kg").
- `calculerPrix()` : retourne le prix estimé, basé sur un tarif de 30 dinars par kilogramme.

Les moutons se répartissent en deux grandes catégories :

- 1) Moutons locaux : Ce sont des moutons qui comprennent des races tunisiennes telles que :

- Barbarine : race rustique bien adaptée aux zones arides.
- Noire de Thibar : race connue pour la qualité de sa laine noire.

Ces moutons peuvent être Bio (élevés sans produits chimiques) ou Non bio.

Cette classe redéfinit les méthodes :

- `toString()` : pour décrire un mouton et préciser s'il est bio ou non.
- `calculerPrix()` : pour ajouter 200 dinars au prix de base si le mouton est bio.

- 2) Moutons importés : Ce sont des moutons qui proviennent de pays étrangers comme l'Espagne, la Roumanie, etc. Cette classe est caractérisée par l'attribut : `paysOrigine` (*chaîne de caractères*) qui précise le pays d'origine du mouton.

Cette classe redéfinit les méthodes :

- `toString()` : pour décrire un mouton et inclure le pays d'origine.
- `calculerPrix()` : pour ajouter une taxe de 25 % sur le prix de base

Travail à faire :

1. Donnez le code nécessaire des classes 'Mouton' (1,5pts), 'MoutonLocal' (1.5pts) et 'MoutonImporte' (1.5pts) en JAVA.

```
// classe Mouton
public class Mouton { // attributs 0.5 pt
    protected String nom;
    protected String race;
    protected double poids;

    public Mouton(String nom, String race, double poids) {
        this.nom = nom;
        this.race = race;
        this.poids = poids;
    }

    public Mouton() { // constructeur par défaut 0.25 pt
        this("Mouton", "Race", 0.0); /* proposition, ils peuvent utiliser un seul construc-
        teur et initialiser les attributs avec des valeurs par défaut */
    }

    public String toString() { // 0.5 pt
        return nom + " : " + race + " de " + poids + " kg";
    }

    public double calculerPrix() { // 0.25 pt
        return poids * 30;
    }
}

// classe MoutonLocal
public class MoutonLocal extends Mouton { // 0.25 pour le mot extends
    private boolean estBio; // 0.25 pt

    public MoutonLocal(String nom, String race, double poids, boolean estBio) {
        super(nom, race, poids);
        this.estBio = estBio;
    }

    @Override
    public String toString() { // 0.5 pt
        return super.toString() + " (Local, " + (estBio ? "Bio" : "Non bio") + ")";
    }

    @Override
    public double calculerPrix() { // 0.5 pt
        double prix = super.calculerPrix();
        if (estBio) {
            prix += 200;
        }
        return prix;
    }
}

// classe MoutonImporte
public class MoutonImporte extends Mouton { // 0.25 pour le mot extends
    private String paysOrigine; // 0.25 pt

    public MoutonImporte(String nom, String race, double poids, String paysOrigine) {
```

```

        super(nom, race, poids);
        this.paysOrigine = paysOrigine;
    }

    @Override
    public String toString() { //0.5 pt
        return super.toString() + " (Importé de " + paysOrigine + ")";
    }

    @Override
    public double calculerPrix() { //0.5 pt
        double prix = super.calculerPrix();
        return prix * 1.25; // Ajout de 25%
    }
}

```

2. Donnez le code Java d'une classe 'Ferme' (2pts) qui :

- Gère un tableau de moutons.
- Lors de la création de la ferme, ce tableau contient :
 - 100 moutons locaux **bio** :
 - 50 de race Barbarine
 - 50 de race Noire de Thibar
 - 100 moutons locaux **non bio** :
 - 50 Barbarine
 - 50 Noire de Thibar
 - 50 moutons **importés** de différents pays

Cette classe contient un **constructeur par défaut** et les **méthodes** suivantes :

- ajouterMouton(Mouton m) : ajoute un mouton à la ferme.
- afficherTousLesMoutons() : affiche la description et le prix estimé de chaque mouton.
- CalculerPrixMoutons() : calcule le prix total des moutons de la ferme.

```

public class Ferme {
    private Mouton[] moutons;
    private int compteur; //0.25 pt pour les attributs

    public Ferme() { //0.5 pt pour le constructeur par défaut
        moutons = new Mouton[250];
        compteur = 0;
        Random rand = new Random();

        // 100 moutons locaux bio
        for (int i = 0; i < 50; i++) {
            ajouterMouton(new MoutonLocal("B" + i, "Barbarine", 40+ rand.nextDouble()*20,
true));
            ajouterMouton(new MoutonLocal("T" + i, "Noire de Thibar", 40+
rand.nextDouble()*20, true));
        }

        // 100 moutons locaux non bio
        for (int i = 0; i < 50; i++) {

```



```

        ajouterMouton(new MoutonLocal("NB" + i, "Barbarine", 40 + rand.nextDouble()*20,
false));
        ajouterMouton(new MoutonLocal("NT" + i, "Noire de Thibar",
40 + rand.nextDouble()*20, false));
    }

    // 50 moutons importés
    String[] pays = {"Espagne", "Roumanie", "Algérie"};
    for (int i = 0; i < 50; i++) {
        ajouterMouton(new MoutonImporte("IMP" + i, "RaceImportée",
40 + rand.nextDouble()*20, pays[rand.nextInt(pays.length)]));
    }
}

public void ajouterMouton(Mouton m) { // 0.5 pt
    if (compteur < moutons.length) {
        moutons[compteur++] = m;
    } else {
        System.out.println("La ferme est pleine !");
    }
}

public void afficherTousLesMoutons() { // 0.5 pt
    for (int i = 0; i < compteur; i++) {
        System.out.println(moutons[i].toString() + " - Prix : " + moutons[i].calculerPrix() + " DT");
    }
}

public double calculerPrixMoutons() { // 0.25 pt
    double total = 0;
    for (int i = 0; i < compteur; i++) {
        total += moutons[i].calculerPrix();
    }
    return total;
}
}

```

3. Que se passe-t-il si on appelle calculerPrix() sur un objet Mouton, mais que c'est un MoutonLocal ? Quelle version de calculerPrix() est utilisée ? Expliquez. (1pt)

Lorsqu'on appelle la méthode calculerPrix() sur un objet déclaré de type Mouton mais instancié en tant que MoutonLocal, c'est la méthode redéfinie dans la classe MoutonLocal qui est exécutée (0.5 pt). Cela est dû au **polymorphisme en Java**, qui fait que **la méthode appelée dépend du type réel de l'objet en mémoire (type d'exécution)**, et non du type de la variable de référence. Ainsi, même si l'objet est manipulé via une variable de type Mouton, la méthode calculerPrix() spécifique à MoutonLocal s'exécutera, prenant en compte, par exemple, s'il est bio ou non. (0.5 pt pour l'explication)

Annexe

Tous les conteneurs contiennent les méthodes suivantes:

size_t size()	Retourne le nombre d'éléments contenus dans le conteneur
bool empty()	Vérifie si le conteneur est vide
iterator begin()	Retourne un itérateur positionné sur le premier élément du conteneur
iterator end()	Retourne un itérateur à accès direct positionné après le dernier élément du conteneur
iterator rbegin()	Retourne un itérateur inverse positionné sur le dernier élément du conteneur
iterator rend()	Retourne un itérateur inverse positionné sur le premier élément du conteneur
void insert(pos, elem)	Insère une copie de l'élément à position spécifiée (pour set et map, pos n'est qu'une indication d'un point de départ pour la recherche de la position d'insertion)
void erase(pos)	Élimine l'élément se trouvant à la position spécifiée
void clear()	Vide le conteneur

Interface de vector<T>:

vector<T>(int n)	Constructeur qui fixe la capacité initiale à <i>n</i>
size_t capacity()	Retourne le nombre maximal d'éléments que le vecteur peut contenir sans avoir besoin d'une nouvelle réallocation
void reserve(n)	Fixe la capacité à <i>n</i> éléments
T& front()	Retourne le premier élément
T& back()	Retourne le dernier élément
T& operator[]	Retourne le <i>nième</i> élément
void push_back(elem)	Insère une copie de l'élément à la fin
void pop_back()	Retire le dernier élément
size_t size()	Retourne la taille du vecteur (c'est-à-dire le nombre d'éléments qu'il contient)
void resize(n)	Fixe la taille à <i>n</i> (si la taille augmente, les espaces supplémentaires sont remplis par les constructeurs par défaut)